

PP05: Deep learning for rapid classification of supersymmetric models at the LHC

Supervisor: Professor Alan Barr

Abstract

This report describes the construction and testing of two neural networks used to predict the CL_s value of supersymmetric models of particle physics, for various existing ATLAS search analyses. A model is categorised as ‘excluded’ if $CL_s < 0.05$ and ‘not excluded’ if $CL_s > 0.05$. The first neural network can categorise 45% of models and the second 29%, whilst each incorrectly categorising fewer than 1% of all models. This corresponds to saving over 150,000 hours of compute time (split across multiple processors) and over 7,500 kg of CO_2 .

1 Introduction

The Standard Model of particle physics is one of the most successful scientific theories of all time, but it is incomplete. While it accurately describes all observed particles and their interactions, there are several phenomena which are not explained within the model, including gravity, dark matter, neutrino oscillations, and the matter-antimatter asymmetry. These unexplained phenomena (and other deficiencies) have motivated many different extensions to the Standard Model. One of these extensions, known as supersymmetry (SUSY), is the focus of this report.

The fundamental implication of SUSY is that for every particle in the Standard Model, there exists a corresponding ‘superpartner particle’ (sparticle) with identical mass, charge, colour, and weak isospin but with a spin that differs by $\frac{1}{2}$. Supersymmetry is a symmetry between bosons and fermions: for each boson (particle with integer spin) there is a supersymmetric fermion (with half-integer spin) and vice versa. The lightest of these supersymmetric particles could be a good candidate for dark matter, and the extra symmetry would solve the so-called ‘hierarchy problem’ [1]. These, and other strengths, make SUSY a very attractive extension to the Standard Model.

At the Large Hadron Collider (LHC), high-energy proton-proton collisions have been used to search for evidence of SUSY. Most recently, the second data-taking run at the LHC (‘Run 2’) took place from 2015–2018, which provided enormous amounts of data in which evidence of sparticles might have been found. However, Run 2 provided no evidence of supersymmetry. This left researchers at something of a crossroads: an enormous amount of work has produced no positive results and the next steps are unclear. However, what we can extract from the Run 2 data is more constraints on which SUSY models are possible. This informs how best to search for SUSY in future data-taking runs.

One group doing this analysis is the ATLAS Collaboration. They specifically focus on data produced by ATLAS, the largest detector at the LHC [2]. As part of their data analysis, many thousands of simulated particle collisions are used to evaluate whether a single SUSY model is excluded by the Run 2 data. Performing these particle simulations is computationally expensive: one model takes around 20 hours of real time on the CERN computing grid and produces 1 kg of CO_2 . In their 2024 paper, over 20,000 SUSY models were evaluated in this way, so the total computational cost of this work is substantial [3]. The goal of this work is to use deep learning (a type of machine learning) to predict very quickly

whether a SUSY model is excluded or not excluded by the Run 2 data, and therefore reduce the required time and environmental cost of these analyses.

Key theoretical background is given in Section 2. This includes a brief overview of supersymmetry, deep learning, and how models are excluded (CL_s). Next, the construction and training of the neural networks (and mistakes to avoid) is described in Section 3. The performance of the neural networks is covered in Section 4, and concluding remarks are made in Section 5.

2 Theoretical background

2.1 Supersymmetry and the pMSSM

The most fundamental implication of SUSY is that for each particle we know of, there is a supersymmetric partner particle. These extra particles introduce a suite of new parameters that describe a particular SUSY model such as the sparticle masses, the coupling strength of different sparticles to the Higgs field, and so on. The simplest supersymmetric extension to the Standard Model is known as the Minimal Supersymmetric Standard Model (MSSM). It introduces the fewest possible new particles and interactions that would result in viable supersymmetry, but it still introduces over 100 new parameters [4].

This huge parameter space is too large to explore in practice, so one approach to making the problem tractable is to impose some constraints on any potential SUSY model:

- no new source of CP-violation.
- no flavour-changing neutral currents.
- first and second generation sfermion universality.

Making these assumptions reduces the number of independent parameters from over 100 to only 19. As these constraints are based on observed phenomena, this simpler version is known as the phenomenological MSSM (pMSSM). More detail on the pMSSM and the origin of the constraints can be found in the original reference [4]. This work exists entirely inside the framework of the pMSSM.

2.2 Understanding CL_s

No matter which extension to the Standard Model turns out to be correct, any new theoretical model of particle physics must be compatible with existing observations. If a new model is incompatible with existing observations, it must be wrong and can be discarded. It is therefore useful to quantify how well a model agrees with existing observations, which is done in this context with one value: CL_s .

The CL_s value of a model takes values between 0 and 1, and is closer to 0 the more a model deviates from observed results. In this context, a model is generally considered excluded if $CL_s < 0.05$. It is defined as

$$CL_s \equiv \frac{CL_{s+b}}{CL_b},$$

where:

- CL_{s+b} is the probability of obtaining a result at least as extreme as the observed result if the SUSY model is correct.
- CL_b is the probability of obtaining a result at least as extreme as the observed result if there is no SUSY signal and the Standard Model is correct.

If a model has a low CL_s value, it means CL_{s+b} (the probability of getting the observed data under the SUSY model) is much lower than CL_b . In simple terms, the data looks more like what we'd expect if

there were no SUSY signal at all. This suggests the SUSY model does not fit the data well, and can be excluded. The goal of this work is to use deep learning to predict the CL_s value of a model given only the model’s parameters.

This is an extremely simple explanation of the CL_s method. More detail can be found in the original reference [5].

2.3 Overview of deep learning

Deep learning is a very active area of research. It is the technology that underpins large language models (like ChatGPT), computer vision systems, modern financial fraud detection, and much more. Its power lies in its ability to discover complex patterns in data. The goal of this work is to find patterns in the pMSSM parameter space that make it possible to predict a model’s CL_s value, so deep learning is a natural tool for this problem.

Deep learning is a type of machine learning, in which a computer can learn to do a task by seeing lots of training examples. It is built on neural networks (NNs). NNs are comprised of layers of nodes, including an input layer, several hidden layers, and an output layer. Each node connects to many nodes in the following layer, and each connection has an associated weight and bias. These are simply numbers that control how strongly one node affects another, and collectively the weights and biases are called the *parameters* of a NN. The parameters are updated in small steps to more closely match outputs to target values – this is what we mean by a NN ‘learning’. Over time, the patterns in the data are encoded into the values of these parameters. Cutting-edge NNs like ChatGPT-4 have over 1 trillion parameters which allow them to learn extremely complex patterns [7].

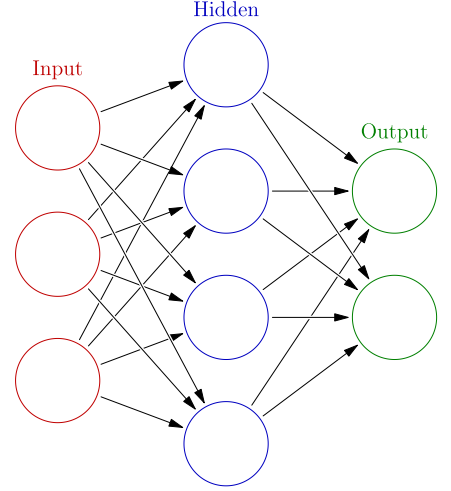


Figure 1: Visualisation of a simple neural network. Each circle represents a node and each line a connection [6].

In the simplest kind of NN, data flows from the input layer, through each of the hidden layers, to the output layer. This is known as a ‘feed-forward’ NN and is the type of NN used in this work. A visualisation of a very simple NN is shown in Figure 1.

2.4 How and why we look for SUSY

Before continuing onto the original work of this paper, it is worth thinking about the big picture: what are we looking for, how do we look for it, and what motivates our search? These questions are briefly discussed here.

Particle colliders like the LHC are built to help us understand how the universe works. This means two things: making precise measurements of things we know exist (like particle masses and lifetimes), and testing new theories of physics. A theory with no positive results like SUSY (at least as of April 2025) becomes less attractive over time, but it is worthwhile extracting what information we can from the data already gathered because it helps point future research in the right direction.

So how do we test new theories at the LHC? The approach is the same as for any scientific theory: we see what predictions a theory makes, and compare those predictions with data. For new models of particle physics, the predictions are made by simulating many particle collisions assuming the new model is correct. After simulating thousands of collisions, the predictions of the model are essentially the rates of certain events. These rates are then compared with what we actually observe and the model can be discarded or explored further.

To understand this better, let’s look at a concrete example from the LHC. One search targets the electroweak production of two specific sparticles, the $\tilde{\chi}_1^\pm$ and $\tilde{\chi}_2^0$, and their leptonic decay via W and Z bosons [8]. The targeted decay chain is

$$\tilde{\chi}_1^\pm + \tilde{\chi}_2^0 \longrightarrow W^\pm \tilde{\chi}_1^0 + Z \tilde{\chi}_1^0 \longrightarrow \ell^\pm \nu \tilde{\chi}_1^0 + \ell^+ \ell^- \tilde{\chi}_1^0,$$

where $\tilde{\chi}_1^0$ is the lightest supersymmetric particle (LSP). The experimental signature of this decay chain is three leptons (two of which have the same flavour but opposite signs), little to no hadronic activity, and large missing transverse momentum¹ due to the two LSPs that escape the detector. There are ways to reach this signature final state within the Standard Model, but if SUSY is correct then we would expect to see this signature more often than SM processes alone could account for. The SM processes are the background, and an excess well above the estimated background is the SUSY signal that the search is looking for.

If a particular SUSY model predicts a large signal and data from the LHC shows no signal, that model can be excluded. Whether or not a model is excluded by observed data is exactly what CL_s measures in this context.

The goal of the ATLAS Collaboration’s work that this report builds on is to find pMSSM models that do not contradict existing observations but which could, with future searches at the LHC, be distinguished from the Standard Model [3]. Their approach involves several steps: choose a pMSSM model by randomly sampling the 19-dimensional parameter space, use simulations to predict the rate of certain signals (such as the signature described above), and compare to observations. The simulation step is very computationally intensive. If some regions of parameter space can be confidently categorised as excluded or not excluded without doing the simulations, it would save enormous amounts of processing time in the search for valid and testable models. This work aims to do exactly this with neural networks.

3 Construction and training of neural networks

3.1 Training data

The training data used here comes in the form of two datasets created by the ATLAS Collaboration in their analysis of electroweak production of supersymmetric particles [3]. For the first dataset, 12,280 pMSSM models were generated by choosing random values for each parameter from a uniform distribution. The second dataset is similar, except it contains 8,897 models and only includes models with a bino-like LSP (i.e. where the LSP is mostly composed of a particular sparticle called the bino). These will be called the EWKino and Bino datasets respectively, matching the naming convention used by the ATLAS Collaboration. A separate NN is trained for each dataset.

Only 12 of the 19 parameters are relevant to electroweak production, so these parameters were randomly generated and the remaining 7 were set to a constant value. Because a constant value has no predictive power, only the 12 relevant parameters are used here as inputs to the NN. The meanings of each of the 19 parameters are summarised in Table 1 in the Appendix.

Each model in the datasets went through many stages of processing, including a very computationally expensive particle simulation stage (for more details see Section 2.1 of the ATLAS Collaboration’s paper [3]). The final output of the process is the CL_s of a model, and therefore whether that model is excluded by any existing observations ($CL_s < 0.05$) or not.

¹In the plane perpendicular to the beam direction, the total particle momentum is zero. A non-zero measurement of this ‘transverse momentum’ after a collision indicates some particles have escaped the detector.

In fact, 11 different CL_s values were computed for each model, each based on a different search such as the search described in Section 2.4. The most restrictive (the lowest) was taken as the final CL_s value. Initially, only the final CL_s is predicted, but later a separate neural network is trained for each CL_s type.

From each dataset, 85% of models were randomly selected as training and validation data. The results in Section 4 are produced using the remaining 15%, which are not seen during training to allow for an independent evaluation of the performance.

3.2 Network architecture

As described in Section 2.3, a neural network is made up of several layers, each of which contains many nodes. The number of layers, the type of each layer, the number of nodes, and how the layers are connected all make up the *architecture* of a neural network. Different architectures are suitable for different tasks: a large language model like ChatGPT and a facial recognition system are both neural networks, but they have very different architectures.

The neural networks trained for the EWKino and Bino datasets have the same architecture: both have 4 hidden layers of 128 nodes each. Each hidden layer is densely connected, meaning every node in one layer is connected to every node in the following layer. Each of the 12 input parameters is normalised to have a mean of 0 and standard deviation of 1 across the dataset, ensuring each input contributes on the same scale. The output of the neural net is a single node scaled to the range (0, 1) with the sigmoid function. This scaling ensures the output is always a valid CL_s value. The network architecture is visualised in Figure 2.

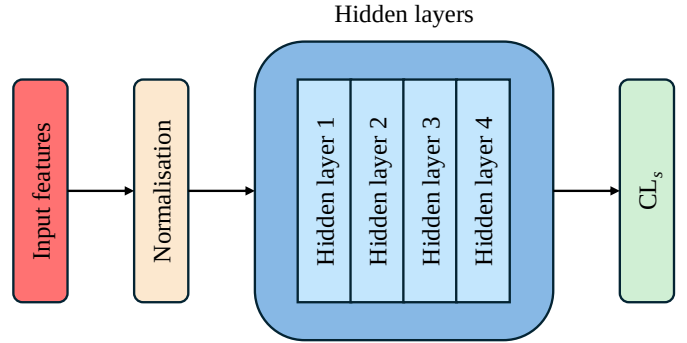


Figure 2: Network architecture for the CL_s predictor. There are 12 input features, and each hidden layer consists of 128 densely-connected nodes. This gives 101,632 trainable parameters.

Alternative architectures have also been tried, including 2 hidden layers of 512 nodes, 6 layers of 32 nodes, and many more. The chosen architecture was found to perform the best. Some technical details: the loss function used is mean absolute error, the activation function of each hidden layer is ReLU, and the activation function of the output node is the sigmoid. Alternative loss functions and output activation functions have been tested, which is described in the following section.

3.3 Misadventures

There are several ideas that seem like they might improve the performance of the NN, but in fact make it worse. They are discussed here.

3.3.1 Custom loss functions

CL_s can take any value between 0 and 1, but the key question is whether CL_s is above or below 0.05. This suggests it might be possible to extract better performance from the NN by weighting models with a low CL_s value more heavily. For example, mistaking 0.025 (invalid) for 0.075 (valid) would be punished more harshly than mistaking 0.925 for 0.975 (both valid). This could result in lower errors for models close to the 0.05 boundary, and therefore better classifying power.

This can be done with a custom loss function. A loss function tells a NN how well it is performing. It takes as input two sets of values: the target (true) values, and the NN’s predictions. Well-known examples include mean squared error and mean absolute error.

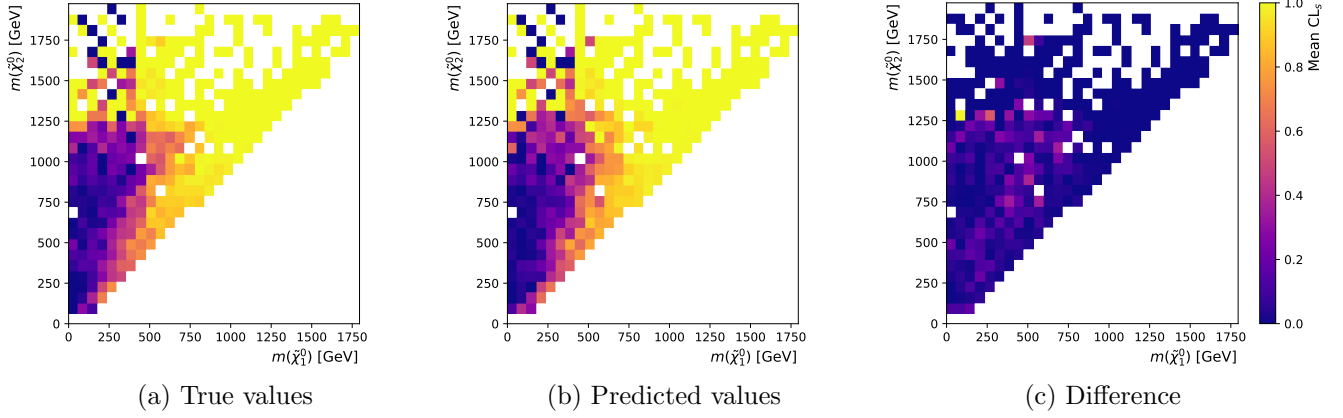


Figure 3: Heatmaps of the mean CL_s in bins of $m(\tilde{\chi}_1^0)$ and $m(\tilde{\chi}_2^0)$ for the EWKino dataset. Shown here are the (a) true values, (b) values predicted by my neural net, and (c) the difference between them. The differences in (c) are mostly close to 0, which suggests the NN performs well. The same colourbar is used for each heatmap and is shown on the far right.

Several custom loss functions were tried, and the best-performing was mean absolute error (MAE) weighted linearly towards zero such that the loss L for n models is given by

$$L = \frac{1}{n} \sum_{i=1}^n |y_i - x_i| \cdot \underbrace{(1 + (1 - x_i))}_{\text{weight}},$$

where x_i and y_i are the i^{th} true and predicted values respectively. Thus models with a true CL_s of 0 will contribute twice as much to the overall error as models with a true CL_s of 1, so the NN should learn to predict these models better. As intended, this reduces the NN’s error for models with a low target CL_s as shown in Figure 8 in the Appendix.

Despite reducing error for models with a low true CL_s , this NN turns out to be a worse classifier. This is because it is useful to correctly categorise both models that are excluded and models that are not excluded, and weighting the NN to focus on low- CL_s models harms the NN’s performance in categorising high- CL_s models.

3.3.2 Binary classifier

In the context of machine learning, the task of predicting a continuous numerical value like CL_s is known as a regression task. Another common task is to sort inputs into categories (such as ‘spam’ and ‘not spam’ for emails), which is known as a classification problem. As the final goal here is to classify models as ‘excluded’ or ‘not excluded’, it is tempting to reframe this problem as a classification problem.

A neural network was trained using this approach and the performance was found to be worse. This is because in a classification problem, the labels you are trying to predict must be discrete – in this case, that means mapping $CL_s < 0.05 \rightarrow 0$ (excluded) and $CL_s > 0.05 \rightarrow 1$ (not excluded). This reduces the amount of information available to the NN and so results in less nuanced predictions.

4 Results: Performance of neural net

4.1 Preliminary results

The most natural way to test the performance of these NNs is to create the same plot using both the true and predicted CL_s values, and to compare the two. An example using the EWKino NN is shown in

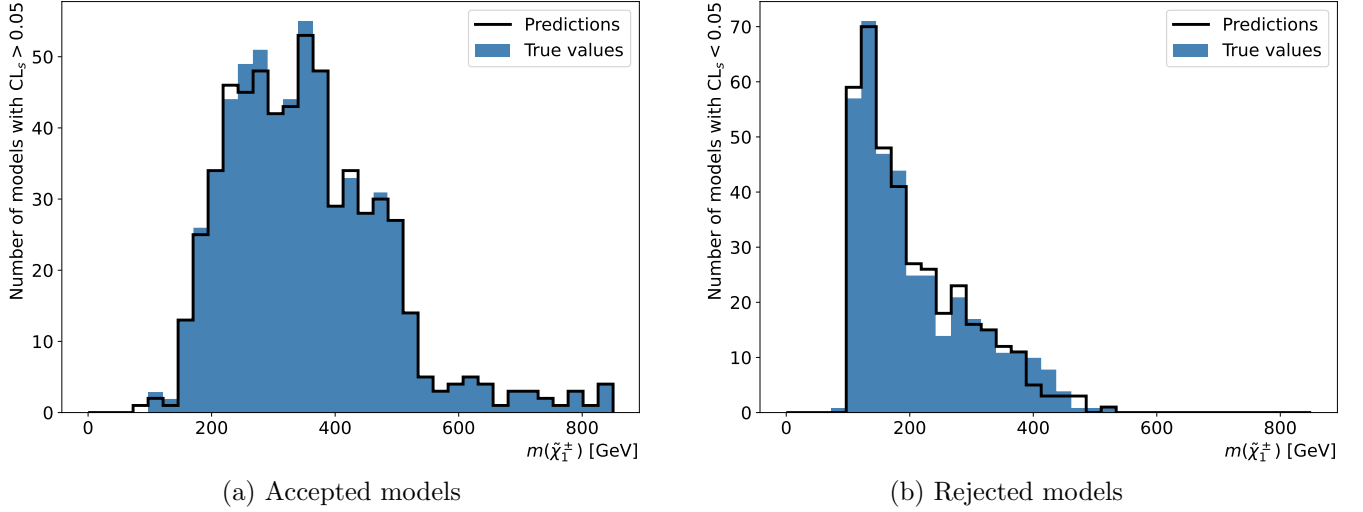


Figure 4: Bar graph of number of (a) accepted and (b) rejected models against $m(\tilde{\chi}_1^\pm)$, the mass of the lightest chargino. The true values are shown in blue and the predicted values in black. In each case the true and predicted values are similar, suggesting the Bino NN is performing well.

Figure 3. The plot is a heatmap of mean CL_s in bins of $m(\tilde{\chi}_1^0)$ and $m(\tilde{\chi}_2^0)$, the masses of the lightest and second-lightest neutralinos respectively. The heatmap of differences is mostly close to 0, suggesting the NN is performing well.

A different plot using the Bino NN is shown in Figure 4. It is a bar graph of the number of accepted and rejected models in bins of $m(\tilde{\chi}_1^\pm)$, the mass of the lightest chargino. The true and predicted values are similar, suggesting the Bino NN is also performing well across a wide range of pMSSM models.

4.2 Performance as a classifier

As discussed above, the purpose of this NN is to reduce the reliance on particle simulations when determining CL_s for pMSSM models. This is done by predicting the CL_s for each model with the NN then classifying each as ‘excluded’, ‘not excluded’, or ‘unsure’ depending on cutoff values that must be chosen. Only those in the ‘unsure’ category will then go through the particle simulation step. Thus the goal is to maximise the number of models categorised as ‘excluded’ or ‘not excluded’ whilst making the fewest possible errors.²

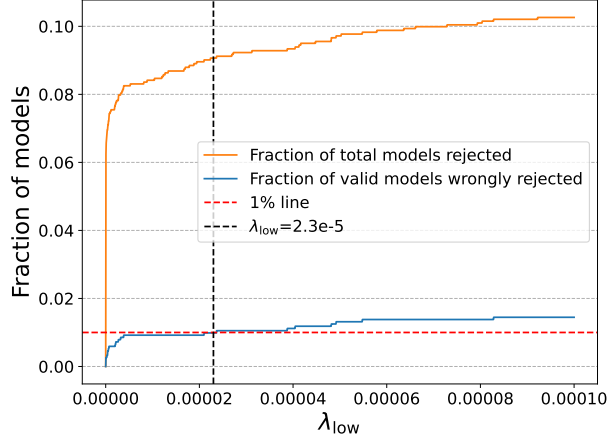
The cutoff values will be called λ_{high} and λ_{low} respectively. Models are categorised as follows:

$$\begin{aligned} CL_s^{\text{pred.}} < \lambda_{\text{low}} &\Rightarrow \text{excluded} \\ CL_s^{\text{pred.}} > \lambda_{\text{high}} &\Rightarrow \text{not excluded} \\ \lambda_{\text{low}} \leq CL_s^{\text{pred.}} \leq \lambda_{\text{high}} &\Rightarrow \text{requires particle simulations} \end{aligned}$$

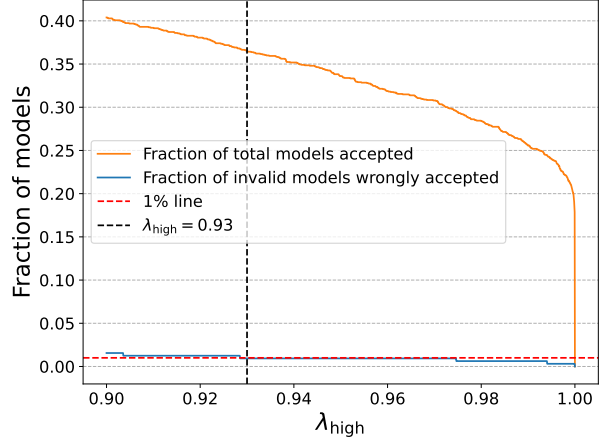
The values of λ_{high} and λ_{low} could be chosen in many ways. Here, they are chosen to satisfy the arbitrary goal of incorrectly categorising fewer than 1% of models. This is a rather stringent requirement and could be relaxed to increase the number of models categorised by the NN. Graphs of model exclusion against λ_{high} and λ_{low} for the EWKino and Bino NN are shown in Figure 5.

With $\lambda_{\text{low}} = 2.3 \times 10^{-5}$ and $\lambda_{\text{high}} = 0.93$, we can categorise 45% of EWKino models as excluded or not excluded whilst incorrectly categorising fewer than 1% of models. For the 12,280 models in this dataset,

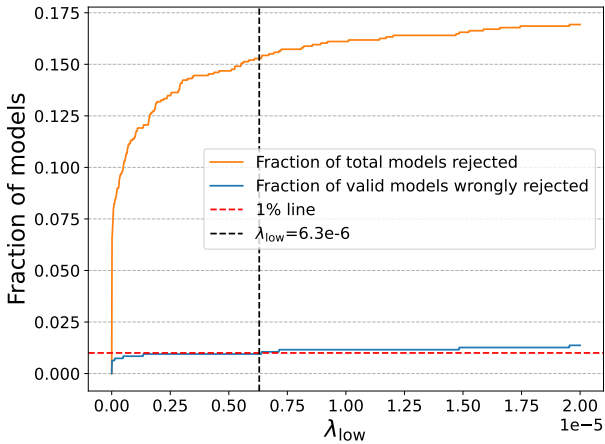
²To help with readability, the categories ‘excluded’ / ‘not excluded’ are also sometimes referred to as ‘invalid’ / ‘valid’ or ‘rejected’ / ‘accepted’.



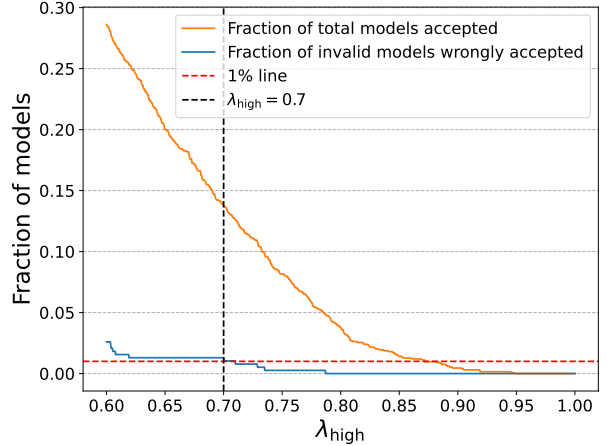
(a) EWKino NN



(b) EWKino NN



(c) Bino NN



(d) Bino NN

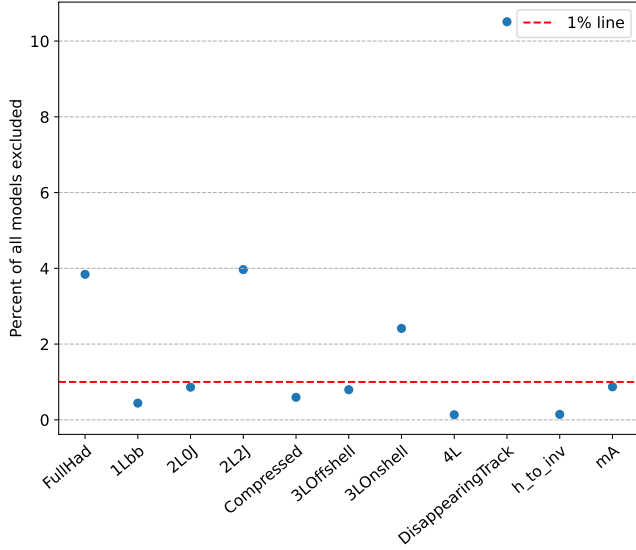
Figure 5: Graphs showing the fraction of models categorised as (a), (c) rejected and (b), (d) accepted as a function of λ_{low} and λ_{high} respectively, for each NN. The fraction of all models categorised is plotted along with the fraction of models incorrectly categorised in each case. For the EWKino NN, $\lambda_{\text{low}} = 2.3 \times 10^{-5}$ and $\lambda_{\text{high}} = 0.93$ results in 45% of models being categorised whilst incorrectly categorising just under 1% of all models. For the Bino NN, $\lambda_{\text{low}} = 6.3 \times 10^{-6}$ and $\lambda_{\text{high}} = 0.7$ results in 29% of models being categorised whilst incorrectly categorising just under 1% of all models. This equates to saving over 150,000 hours of compute time and 7,500 kg of CO_2 .

this would save over 100,000 hours of compute time (split across many processors) and over 5,000 kg of CO_2 .

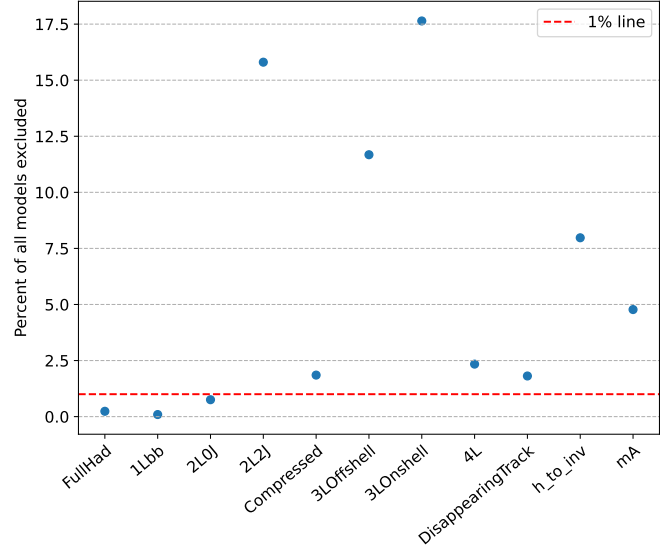
With $\lambda_{\text{low}} = 6.3 \times 10^{-6}$ and $\lambda_{\text{high}} = 0.7$, we can categorise 29% of Bino models whilst incorrectly categorising fewer than 1%. For the 8,897 models in this dataset, this would save over 50,000 hours of compute time and over 2,500 kg of CO_2 .

4.3 Predicting individual CL_s types

The EWKino and Bino NN are trained to predict the so-called ‘Final CL_s ’ of each model. In the ATLAS Collaboration’s paper, 11 different CL_s values were calculated for each model based on 11 different



(a) EWKino dataset



(b) Bino dataset

Figure 6: Scatter plot of the fraction of models excluded by each CL_s type, for the (a) EWKino and (b) Bino dataset. This shows that the same search can have very different excluding power based on which part of the pMSSM parameter space is being analysed.

searches, and the lowest (the strongest constraint) was taken as the Final CL_s. It is interesting to consider the different CL_s types separately.

First, we should note that three of the CL_s types (DisappearingTrack, h_to_inv, and mA) are not technically CL_s values, since they only take values of 0 (excluded) or 1 (not excluded). As these are discrete labels, a dedicated NN trained as a binary classifier with the binary cross-entropy (BCE) loss function is more suitable than the MAE loss function.

Second, it is instructive to visualise the excluding power of each CL_s type. Figure 6 shows scatter plots of the percentage of models excluded in the EWKino and Bino datasets by each CL_s type. The two datasets show very different results: for the EWKino dataset, DisappearingTrack excludes the most models at just over 10%. For the Bino dataset, all of 3LOnshell, 2L2J, and 3LOffshell exclude over 10% and DisappearingTrack excludes just over 1%. This demonstrates how different parts of the pMSSM parameter space show very different phenomenology and a variety of searches are needed to explore the full parameter space.

A separate NN was trained to predict each CL_s type for the EWKino dataset. These NNs all had 4 hidden layers of 128 nodes, but the three CL_s types with discrete labels were trained with BCE rather than MAE as the loss function. And here, a limitation is found: as seen in Figure 7, the DisappearingTrack NN can perfectly model the training data before overfitting. This suggests more data is required to

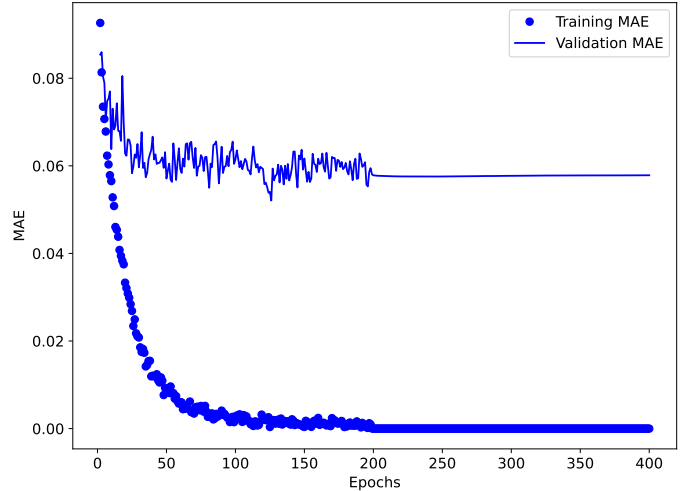


Figure 7: Graph of training and validation MAE against epoch for a NN specifically for DisappearingTrack CL_s. The loss function used in training is BCE, while the loss function used as a metric is MAE. The training MAE reaches 0 before overfitting is seen, which means more data is required to see further improvement.

see improvements in the performance of this NN. Given how large a fraction of Final CL_s comes from DisappearingTrack for the EWKino dataset, this may also be limiting how well the EWKino NN can perform.

5 Conclusion and next steps

This report presents the construction and testing of two neural networks capable of predicting the CL_s values of pMSSM models for various existing ATLAS search analyses with high accuracy. The EWKino NN categorises 45% of models as excluded or not excluded whilst maintaining an error rate below 1%. The Bino NN categorises 29% of models whilst maintaining the same error rate. The combined effect of both NNs would save over 150,000 hours of compute time and over 7,500 kg of CO₂. This demonstrates that deep learning approaches can reliably identify patterns in the pMSSM parameter space and significantly reduce the computational cost of SUSY searches. It is a scalable approach for future SUSY searches and could help unlock the analysis of higher-dimensional parameter spaces with richer phenomenology. These methods are also applicable to other searches for physics beyond the Standard Model besides SUSY.

There are lots of ways to build on this work. The method could be generalised to explore the full 19-dimensional pMSSM parameter space rather than the 12-dimensional subspace explored here. Specialised NNs could be trained for each type of CL_s , and the predictions of each NN could perhaps be used in combination rather than simply taking the lowest as the final prediction. More sophisticated NN architectures might improve the categorisation rate. In particular, it would be exciting to see physics-informed neural networks (for which physical constraints are embedded into the NN) applied to this problem, as they could perform better even with less training data.

Despite the lack of evidence for SUSY in the Run 2 data, this work demonstrates that machine learning can help make more efficient use of existing data and guide future searches. The methods developed here can be readily applied to future LHC runs (Run 3 will be completed in July 2026), where we may finally see evidence of SUSY or other physics beyond the Standard Model.

References

- [1] S. Martin. *A Supersymmetry Primer*. World Scientific, 07 1998.
- [2] ATLAS Collaboration. The ATLAS Experiment at the CERN Large Hadron Collider. *Journal of Instrumentation*, 3(08):S08003, 08 2008.
- [3] ATLAS Collaboration. ATLAS Run 2 searches for electroweak production of supersymmetric particles interpreted within the pMSSM. *Journal of High Energy Physics*, 05 2024.
- [4] A. Djouadi, S. Rosier-Lees, M. Bezouh, et al. The Minimal Supersymmetric Standard Model: Group Summary Report, 1999.
- [5] A. Read. Modified frequentist analysis of search results (the CL_s method). *CERN Report*, 01 2000.
- [6] Wikipedia User Rayhem. Artificial neural network with layer coloring. https://commons.wikimedia.org/wiki/File:Colored_neural_network.svg, 02 2013. [Online; accessed 18-March-2025].
- [7] D. Patel and G. Wong. GPT-4 Architecture, Infrastructure, Training Dataset, Costs, Vision, MoE. <https://semianalysis.com/2023/07/10/gpt-4-architecture-infrastructure/>, 07 2023. [Online; accessed 15-March-2025].
- [8] CMS Collaboration. Searches for electroweak production of charginos, neutralinos, and sleptons decaying to leptons and W, Z, and Higgs bosons in pp collisions at 8 TeV. *The European Physical Journal C*, 74(9), 09 2014.

A Additional plots and tables

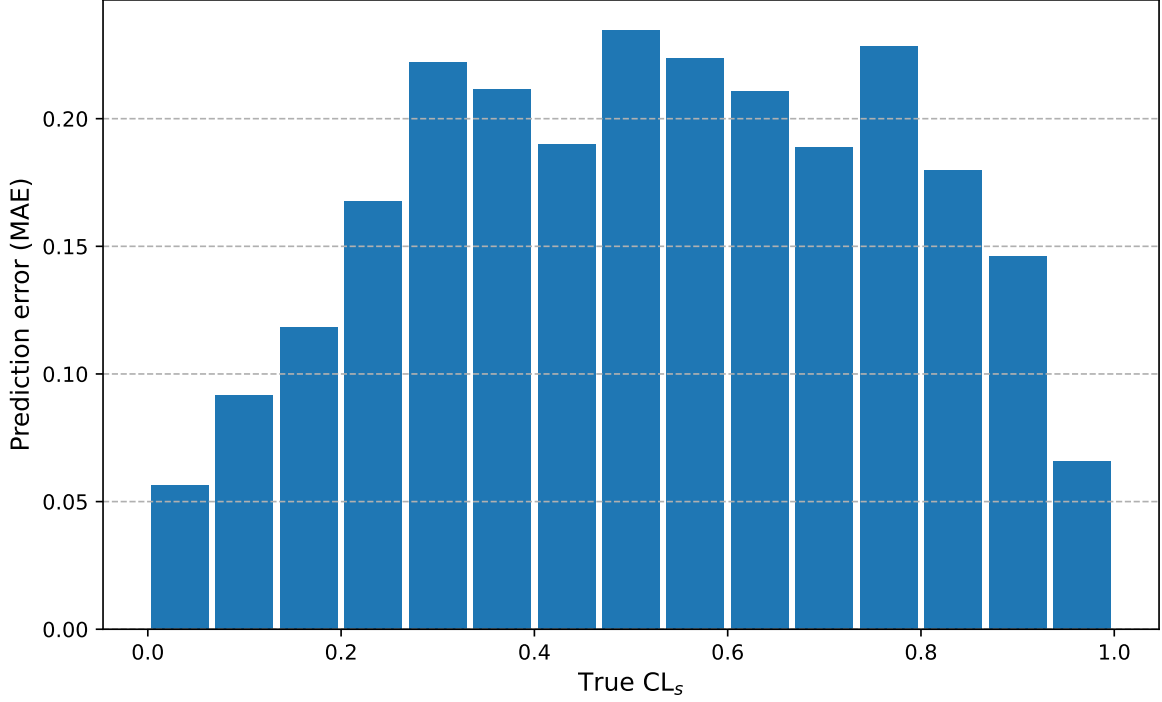


Figure 8: The mean absolute error of the EWKino NN’s predictions in bins of the true CL_s , when the NN is trained with a custom loss function weighted towards 0. As intended, the NN performs better (has smaller errors) for smaller values of true CL_s . However, it was found that the NN performs worse as a classifier when trained with this loss function.

Table 1: Summary of the 19 pMSSM parameters, adapted from Reference [3]. The parameters in red are those set to a constant value and the remainder are used here as inputs to the NNs.

pMSSM Parameter	Meaning
$\tan \beta$	Ratio of the Higgs vacuum expectation values for the two doublets
M_A	Pseudoscalar (CP -odd) Higgs boson mass parameter
μ	Higgsino mass parameter
M_1, M_2, M_3	Bino, wino and gluino mass parameters
A_t, A_b, A_τ	Third generation trilinear couplings
$M_{\tilde{q}}, M_{\tilde{u}_R}, M_{\tilde{d}_R}, M_{\tilde{l}}, M_{\tilde{e}_R}$	First/second generation sfermion mass parameters
$M_{\tilde{Q}}, M_{\tilde{t}_R}, M_{\tilde{b}_R}, M_{\tilde{L}}, M_{\tilde{\tau}_R}$	Third generation sfermion mass parameters